

Table of contents

- [Introduction and motivation](#)
- [Changes](#)
- [Previous polls](#)
- [Intention for wording changes](#)
 - [wait and notify behaviour](#)
 - [Question answered by SG1](#)
 - [Question for LEWG](#)
- [Example](#)
- [Implementation experience](#)
- [Impact on existing code](#)
- [Proposed changes to wording](#)
 - [Feature test macro](#)

atomic and atomic_ref

Introduction and motivation

This paper proposes marking most of `atomic<T>` methods and associated functions `constexpr` to allow usage of atomic code without changes in a `constexpr` and `constexpr` code.

Proposed changes will allow implementing other types (`std::shared_ptr<T>`, persistent data structures with atomic pointers) and algorithms (thread safe data-processing, like scanning data with atomic counter) with just sprinkling `constexpr` to their specification.

Changes

- [R2](#) → R3: Added location of library feature test macro, removed `atomic<shared_ptr>` and `atomic<weak_ptr>`.
- [R1](#) → [R2](#): Added clarification for behaviour of `wait` and `notify` functions.
- [R0](#) → [R1](#): Make `wait` and `notify` functions as requested by SG1. Wording changed accordingly. Updated link to implementation on Compiler Explorer.

Previous polls

SG1: Forward P3309 to LEWG with the following notes:

- Add `constexpr` to the `wait` and `notify` functions in the next revision of P3309
- `atomic<shared_ptr>` should be supported in `constexpr` whenever `shared_ptr` is supported in `constexpr` (whichever paper lands second should have this change)
- `is_lock_free()` should not be made `constexpr`

SF	F	N	A	SA
2	10	4	0	0